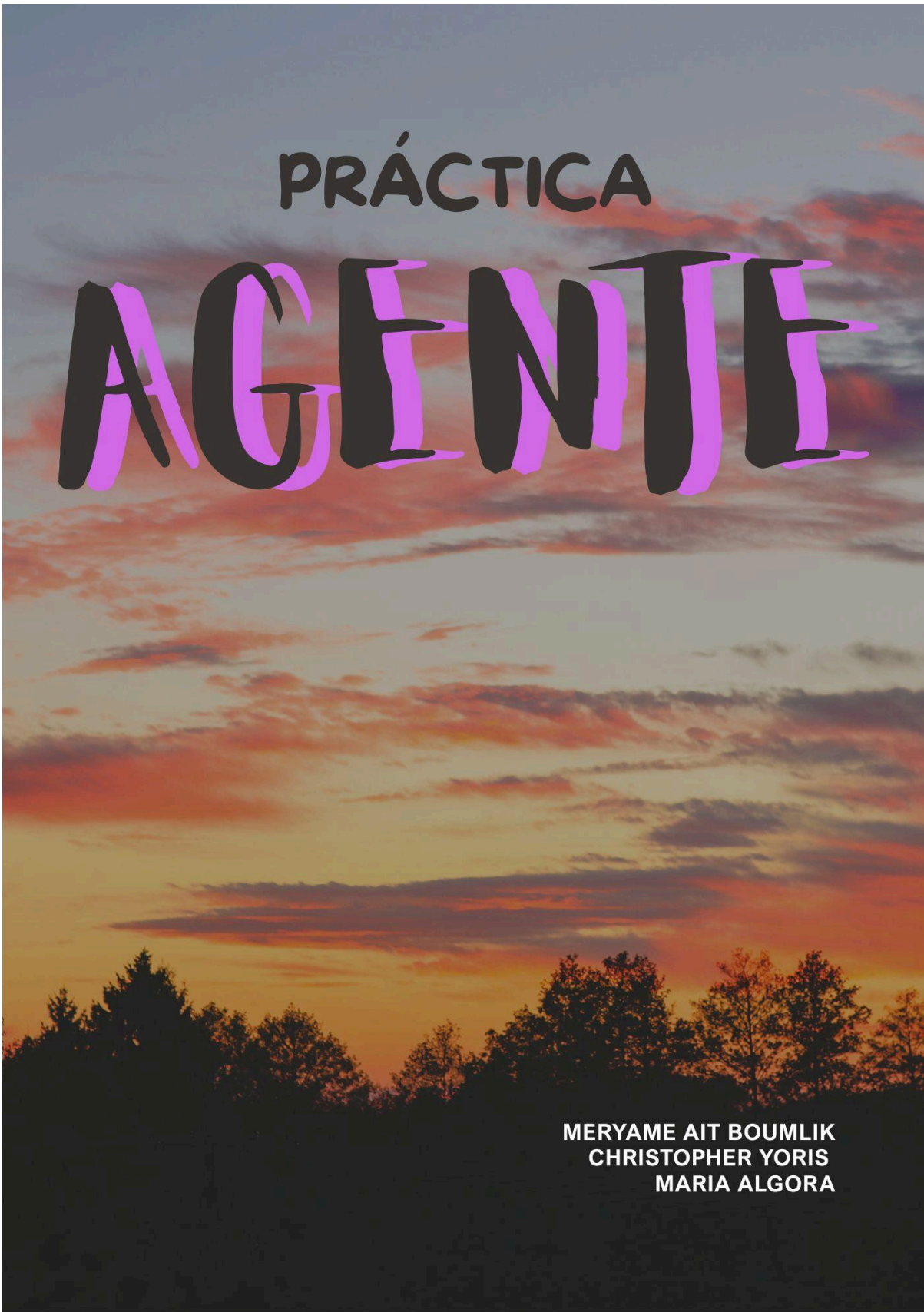




PRÁCTICA AGENTE

MERYAME AIT BOUMLIK
CHRISTOPHER YORIS
MARIA ALGORA

Indice

Indice	2
1. Arquitectura elegida y justificación	3
2. Decisiones tomadas durante el desarrollo	6
3. Flujo detallado de una consulta	8
Paso 1: Entrada y Resolución Dinámica de Entorno	8
Paso 3: Recuperación Semántica de Contexto (Retrieval)	9
Paso 4: Construcción del Prompt Anti-Alucinación	9
Paso 5: Inferencia y Captura de Telemetría	9
Paso 6: Consolidación y Devolución del Contrato	10
4. Resultados de las pruebas realizadas	11
Evaluación mediante RAGAs	11
5. Análisis crítico y limitaciones	12
6. Propuestas de mejora	13
7. Dificultades encontradas y soluciones aplicadas	15

1. Arquitectura elegida y justificación

El propósito de este apartado es detallar la estructura técnica del sistema RAG desarrollado, justificando las decisiones de diseño y analizando su comportamiento real. A lo largo de esta sección, se presenta inicialmente la división modular del agente único y las decisiones clave de su desarrollo. Posteriormente, se describe el recorrido que sigue una consulta dentro del sistema, para finalizar con una evaluación cuantitativa de su rendimiento mediante la librería Ragas, identificando las limitaciones actuales y las soluciones aplicadas ante las dificultades encontradas.

Para resolver este problema, se ha rechazado el diseño de un script único o monolítico en favor de un enfoque modular. En un sistema RAG, cada etapa (la lectura de archivos, la vectorización, la búsqueda en la base de datos y la generación del texto) tiene requerimientos técnicos completamente distintos. Al aislar cada una de estas tareas en un módulo independiente con una responsabilidad única, el sistema gana en flexibilidad. Este diseño modular no solo hace que el código sea más limpio y fácil de auditar, sino que permite experimentar con diferentes configuraciones —como cambiar de base de datos o alternar entre modelos locales y remotos— modificando únicamente el componente afectado y sin alterar la estabilidad del pipeline general.

La organización del código se divide en partes especializadas, todas coordinadas por un administrador central (`pipeline.py`):

- **Punto de Entrada (`consultar.py`):** Es el archivo que inicia la ejecución. Configura las rutas del proyecto y llama al pipeline, separando la interfaz de usuario de la lógica interna.
- **Configuración (`config.py`):** Reúne las variables de entorno y parámetros de los modelos. Permite cambiar fácilmente entre el uso de modelos locales (Ollama) y remotos (PoliGPT).
- **Fragmentación (`chunker.py`):** Carga los documentos de la base de conocimiento y los divide en fragmentos cortos usando `RecursiveCharacterTextSplitter`, guardando siempre el origen y la posición de cada texto.
- **Conversión Vectorial (`embedder.py`):** Se encarga de transformar el texto en vectores (*embeddings*), permitiendo procesar frases sueltas o bloques de texto en grupo.
- **Búsqueda Semántica (`retriever.py`):** Controla la base de datos vectorial en **ChromaDB**. Busca los fragmentos más parecidos a la consulta del usuario y convierte las distancias matemáticas en un puntaje (*score*) de relevancia.
- **Diseño de Prompts (`prompts.py`):** Contiene las plantillas de instrucciones para el modelo. Organiza el contexto recuperado y aplica reglas estrictas para evitar que el modelo invente información (anti-alucinación).
- **Generación de Respuesta (`generator.py`):** Conecta con los modelos de lenguaje (Ollama o PoliGPT) para obtener el texto final. Además, mide el rendimiento del proceso, registrando el tiempo de espera y el consumo de *tokens*.

La separación del sistema en módulos independientes aporta dos ventajas fundamentales para el desarrollo y la auditoría del proyecto. En primer lugar, ofrece una gran facilidad de intercambio tecnológico; al estar las responsabilidades divididas, es posible sustituir la base de datos vectorial o alternar entre diferentes modelos de lenguaje modificando exclusivamente el archivo correspondiente, sin necesidad de alterar la estructura general ni el funcionamiento del pipeline. En segundo lugar, este diseño garantiza la transparencia y la medición detallada de cada consulta, ya que el sistema no se limita a entregar una respuesta abierta, sino que genera un objeto JSON estructurado que expone las fuentes exactas utilizadas, los fragmentos de texto seleccionados y las métricas de velocidad y rendimiento en tiempo real.

El flujo completo de procesamiento, desde que el usuario realiza la consulta hasta que se genera el resultado con sus métricas, se representa en el siguiente diagrama de secuencia:



Esta arquitectura permite mantener una estructura sencilla y comprensible, cumpliendo al mismo tiempo con todos los requisitos funcionales de un sistema RAG basado en un único agente.

2. Decisiones tomadas durante el desarrollo

El desarrollo integra de manera estricta todos los componentes e infraestructuras exigidos en el manual de la práctica. En primer lugar, se ha implementado el almacenamiento de la información mediante el cliente persistente en disco de ChromaDB en los módulos correspondientes. Esto garantiza que el índice vectorial se conserve entre diferentes ejecuciones del programa, eliminando procesos de carga y tokenización redundantes en cada inicio.

En cuanto a la infraestructura de los modelos de lenguaje, el pipeline se ha diseñado para cumplir con la obligatoriedad de operar en local a través del servidor de Ollama (utilizando de manera predeterminada el modelo `llama3.2:3b`), asegurando así el funcionamiento autónomo del sistema. Como complemento a esta conectividad local, el diseño incorpora la compatibilidad con la infraestructura remota de PoliGPT de la UPV mediante una interfaz compatible con OpenAI. Finalmente, la validación del agente se ha alineado con las pautas de evaluación formal de la asignatura mediante la creación de un entorno preparado para el framework RAGAS, permitiendo el cálculo automatizado de métricas de rendimiento y fidelidad.

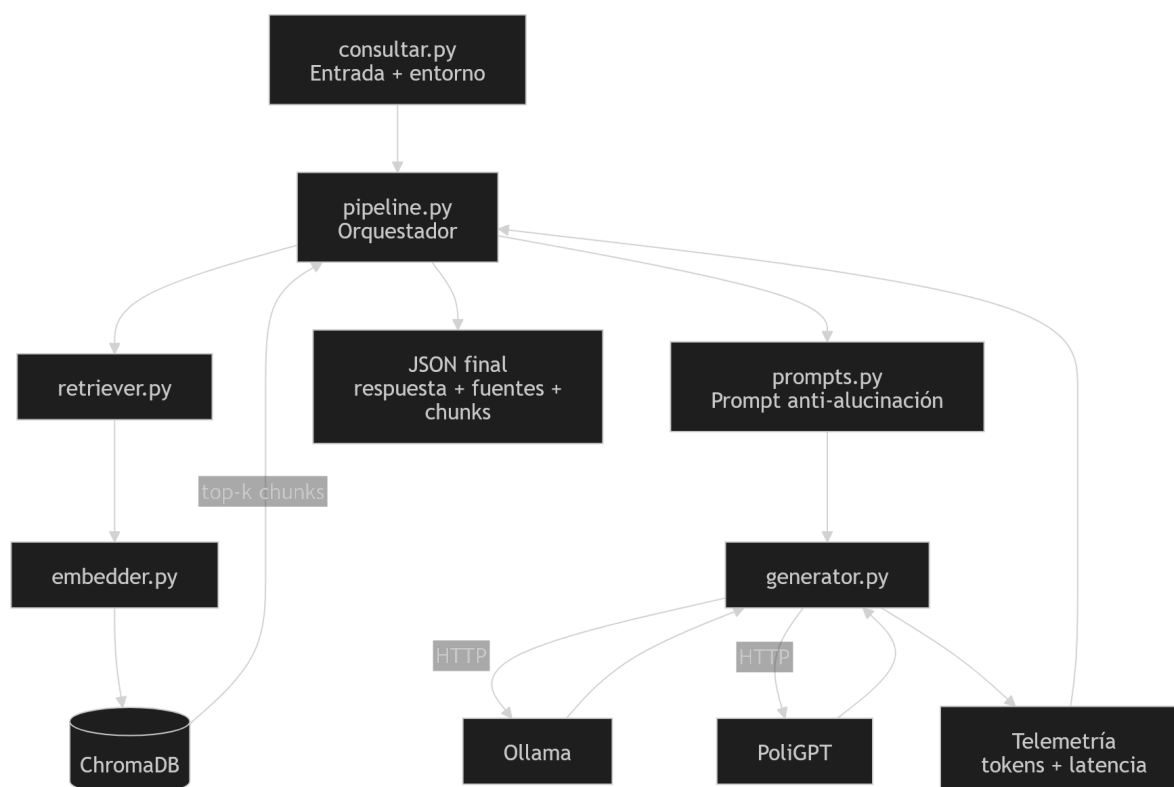
El comportamiento definitivo del pipeline se consolidó mediante la aplicación de soluciones técnicas orientadas exclusivamente a cumplir con los requisitos del proyecto y asegurar el éxito del objetivo planteado:

- **Optimización de la ventana de fragmentación (Chunking):** Aunque las recomendaciones iniciales de la práctica sugerían un tamaño de corte estándar de 500 caracteres, el análisis de la heterogeneidad del corpus —que incluye tablas de horarios y estructuras rígidas de preguntas y respuestas en los FAQ— motivó la decisión de ampliar la ventana a 1000 caracteres con un solapamiento de 250 en `chunker.py`. Este ajuste técnico reduce significativamente el riesgo de truncar la conexión entre una pregunta y su respuesta inmediata dentro de los documentos de texto de la asociación, dotando al modelo de un contexto mucho más rico y cohesionado.
- **Compatibilidad con scripts externos de evaluación y benchmarking:** Una decisión clave en `consultar.py` fue la exposición estratégica de variables globales de configuración (`SERVIDOR_LLM` y `LLM_MODEL`), estableciendo por defecto el entorno local con `llama3.2:3b`. Esta solución de ingeniería se adoptó para garantizar una compatibilidad absoluta con las herramientas de evaluación externa del equipo (como los bancos de pruebas de métricas masivas o scripts de benchmarking), permitiendo modificar dinámicamente el modelo activo al vuelo de forma transparente. Además, el script incluye un mapeo dinámico que asigna alias de ejecución seguros (como transformar la configuración de PoliGPT a `gpt-4o-mini`), protegiendo la estabilidad de la aplicación ante peticiones externas.

- **Control estricto de la respuesta y formato de salida:** La plantilla en [prompts.py](#) fuerza al modelo lingüístico a responder de manera exclusiva con las evidencias del contexto, prohibiendo la invención de datos logísticos (horarios o fechas) e inyectando de forma estricta la constante [REJECTION_PHRASE](#). Esto asegura que ante cualquier ausencia de información el sistema devuelva exactamente la cadena exigida por los tests de no-alucinación: *"No tengo esa información en mis fuentes"*. Asimismo, se estructuró el output final en un formato JSON que expone de forma transparente las fuentes utilizadas, los fragmentos de texto seleccionados y la telemetría de latencia y consumo de tokens gestionada desde el pipeline, asegurando la total audibilidad del proceso.
- Se decidió de manera estratégica implementar el punto de entrada bajo la Opción A (Módulo Python) contemplada en las especificaciones de la práctica. Esta elección técnica simplifica la arquitectura del repositorio y garantiza el éxito de las pruebas automáticas del corrector, exponiendo en la raíz el archivo [consultar.py](#) con la función nativa `consultar(pregunta, conversation_id=None)`. Al invocar directamente el pipeline empaquetado, la función satisface con total precisión el contrato de la práctica al estructurar y retornar un diccionario con las claves obligatorias requeridas para la validación del agente: la respuesta final, la lista de fuentes, los fragmentos o chunks recuperados, las trazas y la telemetría de métricas.

3. Flujo detallado de una consulta

Para comprender la interacción entre los diferentes módulos independientes, es necesario analizar el recorrido secuencial que realiza una consulta desde que es introducida por el usuario o el sistema de evaluación hasta que el agente consolida el resultado estructurado.



Paso 1: Entrada y Resolución Dinámica de Entorno

- **Componente:** `consultar.py` (Raíz del repositorio)
- **Dirección del flujo:** Interfaz del evaluador → Orquestador del Pipeline
- **Proceso:** La ejecución se inicia formalmente mediante la invocación de la función de contrato `consultar(pregunta, conversation_id=None)` o a través de la terminal ejecutando `python consultar.py "<pregunta>"`. En este instante, el módulo intercepta la petición y lee las variables globales de configuración `SERVIDOR_LLM` y `LLM_MODEL`. Aunque el modelo predeterminado en la configuración del sistema es `llama3.2:3b`, el archivo permite alterar este comportamiento inyectando dinámicamente cualquier otro modelo al vuelo.

Paso 2: Orquestación del Pipeline

- **Componente:** `src/agente_rag/pipeline.py`
- **Dirección del flujo:** `consultar.py` → `pipeline.py` (Función `answer()`)
- **Proceso:** Este archivo toma el control absoluto del ciclo de vida de la petición. Actúa como el núcleo del agente único, coordinando de manera estrictamente secuencial la comunicación entre los servicios de extracción de información, inyección de contexto y generación lingüística, recibiendo el modelo activo seleccionado dinámicamente en el paso anterior.

Paso 3: Recuperación Semántica de Contexto (Retrieval)

- **Componente:** `src/agente_rag/retriever.py`
- **Dirección del flujo:** `pipeline.py` → `embedder.py` → **ChromaDB** → `pipeline.py`
- **Proceso:** La pregunta en lenguaje natural es enviada al módulo de extracción. El texto se transforma en un vector numérico mediante el archivo `embedder.py` y se interroga a la base de datos vectorial persistente. El sistema realiza una búsqueda top-k y recupera los fragmentos más cercanos por similitud semántica (parámetro fijo de `k=5`).
- **Estructura de los datos extraídos:** Cada chunk recuperado se empaqueta como un objeto estructurado que conserva sus propiedades esenciales de metadatos:
 - **source:** Nombre exacto del archivo origen del corpus (ej. `07 desayunos logistica.txt`).
 - **text:** Contenido literal extraído del fragmento, cuya longitud viene acotada desde la fase de segmentación en `chunker.py` con un `chunk_size=1000` y `chunk_overlap=250`.
 - **score:** Distancia matemática o puntuación de relevancia normalizada.

Paso 4: Construcción del Prompt Anti-Alucinación

- **Componente:** `src/agente_rag/prompts.py`
- **Dirección del flujo:** Objetos de contexto → Plantilla del Dominio (Asociación DNI)
- **Proceso:** El orquestador unifica los objetos extraídos y los inyecta en la función `build_prompt()`. En esta fase, el contenido se formatea bajo una estructura rígida que redefine el rol del modelo hacia el ámbito de voluntariado de la Asociación DNI Valencia y aplica directrices éticas estrictas: obligatoriedad de responder exclusivamente con la información del contexto, prohibición absoluta de inventar datos numéricos o logísticos (horarios, direcciones, fechas de eventos) e inyección de la constante estricta `REJECTION_PHRASE`. Si la respuesta no existe en el corpus, el modelo es forzado a devolver textualmente la cadena exacta: *"No tengo esa información en mis fuentes"*.

Paso 5: Inferencia y Captura de Telemetría

- **Componente:** `src/agente_rag/generator.py`
- **Dirección del flujo:** Prompt blindado → Servidor de Inferencia (Ollama / PoliGPT)

- **Proceso:** El generador recibe las instrucciones estructuradas y enruta la petición por HTTP según los parámetros validados y pasados dinámicamente. Si la ejecución es local, se conecta con la API de Ollama; si es remota, apunta al endpoint de PoliGPT. Durante el tiempo que dura la inferencia, el componente monitoriza el rendimiento del servidor para estructurar un bloque exhaustivo de telemetría que expone el rendimiento real del modelo utilizado.

Paso 6: Consolidación y Devolución del Contrato

- **Componente:** `consultar.py` (Retorno al evaluador)
- **Dirección del flujo:** `pipeline.py` → Output estructurado en JSON
- **Proceso:** El control regresa al punto de entrada raíz del repositorio. Toda la información recopilada en las fases anteriores se unifica y se formatea en el diccionario de salida exacto exigido por la normativa de la asignatura. El corrector automático recibe el objeto con las claves reales de la implementación, donde las trazas quedan explícitamente parametrizadas como vacías en esta versión:

```
{
  "respuesta": "Texto final generado por el LLM...",
  "fuentes": ["07 desayunos logistica.txt"],
  "chunks": [
    {
      "source": "07 desayunos logistica.txt",
      "text": "Texto literal extraído de la base de datos...",
      "score": 0.1425
    }
  ],
  "metricas": {
    "prompt_tokens": 512,
    "output_tokens": 84,
    "tokens_per_sec": 24.5,
    "latencia_s": 1.24,
    "modelo": "llama3.2:3b"
  },
  "trazas": null,
  "conversation_id": "id_conversacion_o_null"
}
```

4. Resultados de las pruebas realizadas

Benchmark

La evaluación del comportamiento del agente RAG DNI se automatizó mediante el script `benchmark.py`, el cual inyectó secuencialmente cada modelo en el módulo común `consultar.py`, ejecutando un set fijo de preguntas almacenadas en `preguntas.json` y volcando las métricas crudas en `benchmark.json`.

Se evaluaron 4 modelos distribuidos en dos entornos:

- Modelos Locales (vía Ollama): `gemma3:4b` y `llama3.2:3b` ejecutados localmente para medir la viabilidad de un despliegue sin dependencias externas.
- Modelos Remotos (vía PoliGPT API): `poligpt` y `gemma3:27b` requiriendo conexión activa a la VPN de la UPV.

El set de prueba incluyó preguntas de control factuales directas (P1, P3), tareas de síntesis de múltiples documentos (P2, P4) y una pregunta de control fuera de ámbito (P5) para evaluar la resistencia a alucinaciones.

A continuación se consolida la información del postproceso (`benchmark.md`). Las métricas de *tokens de entrada*, *tokens de salida* y *latencia* se extrajeron directamente de los objetos de respuesta (`response.usage` en PoliGPT y `eval_count/duration` en Ollama). La Calidad Subjetiva se determinó bajo el estricto criterio de *Acierto* o *Fallo cognitivo*.

ID	Pregunta	Modelo	Servidor	Latencia (s)	Tokens Salida	Comportamiento Destacado
1	¿Qué es DNI?	gemma3:4b	ollama_local	8.08	102	Respuesta completa y fiel.
1	¿Qué es DNI?	llama3.2:3b	ollama_local	7.07	102	Respuesta completa y fiel.
1	¿Qué es DNI?	poligpt	poligpt	32.82	265	Incluye formato Markdown (negritas).
1	¿Qué es DNI?	gemma3:27b	poligpt	4.42	100	Respuesta estructurada y precisa.
2	¿Cómo me apunto a los desayunos...?	gemma3:4b	ollama_local	2.98	62	Tono cercano ("¡Hola!").
2	¿Cómo me apunto a los desayunos...?	llama3.2:3b	ollama_local	3.50	93	Genera una lista numerada con los pasos.
2	¿Cómo me apunto a los desayunos...?	poligpt	poligpt	61.04	158	Redacción correcta pero latencia alta.
2	¿Cómo me apunto a los desayunos...?	gemma3:27b	poligpt	2.97	61	Precisión directa y rápida.
3	¿Quién paga la gasolina del refuerzo?	gemma3:4b	ollama_local	2.12	24	Respuesta directa y breve.
3	¿Quién paga la gasolina del refuerzo?	llama3.2:3b	ollama_local	1.50	37	Referencia explícita al nombre del archivo.
3	¿Quién paga la gasolina del refuerzo?	poligpt	poligpt	114.27	192	Fiel al contexto pero excesivamente lento.
3	¿Quién paga la gasolina del refuerzo?	gemma3:27b	poligpt	2.51	24	Fiel al contexto con baja latencia.
4	¿En qué se diferencian RESIS y COLES?	gemma3:4b	ollama_local	2.77	63	Sintetiza la diferencia de forma directa.
4	¿En qué se diferencian RESIS y COLES?	llama3.2:3b	ollama_local	2.41	106	Fallo cognitivo: responde con un saludo genérico.
4	¿En qué se diferencian RESIS y COLES?	poligpt	poligpt	136.39	540	Desglose exhaustivo por categorías en Markdown.
4	¿En qué se diferencian RESIS y COLES?	gemma3:27b	poligpt	2.93	62	Extrae información clave sobre los vínculos.
5	¿Cuánto cuesta el alquiler en Valencia?	gemma3:4b	ollama_local	1.84	9	Rechazo estricto exitoso.
5	¿Cuánto cuesta el alquiler en Valencia?	llama3.2:3b	ollama_local	1.13	10	Rechazo estricto exitoso.
5	¿Cuánto cuesta el alquiler en Valencia?	poligpt	poligpt	118.21	166	Rechazo estricto exitoso.
5	¿Cuánto cuesta el alquiler en Valencia?	gemma3:27b	poligpt	2.08	10	Rechazo estricto exitoso.

A. Capacidad de Síntesis e Inferencia Compleja (Pregunta 4)

- **Profundidad vs Rendimiento:** El modelo `poligpt` generó la respuesta más elaborada (540 tokens de salida), dividiendo las diferencias entre RESIS y COLES por público, actividad, entorno y objetivo. Sin embargo, el modelo `llama3.2:3b` falló en la tarea de síntesis, devolviendo un saludo genérico ("¡Hola! Me alegra ayudarte...") y una lista de archivos sin contestar a la pregunta.
- **El equilibrio local:** `gemma3:4b` logró responder correctamente sintetizando que RESIS es para ancianos y COLES para niños en situación vulnerable utilizando solo 63 tokens de salida.

B. Adherencia al Prompt y Anti-Alucinación (Pregunta 5)

- Todos los modelos sin excepción (`gemma3:4b`, `llama3.2:3b`, `poligpt` y `gemma3:27b`) pasaron la prueba de control.
- Ante la pregunta sobre el alquiler, los cuatro modelos generaron exactamente la misma cadena de texto de rechazo: "No tengo esa información en mis fuentes."

C. Análisis de Latencia Crítica

- **Degradación del servidor PoliGPT:** Se observan latencias anómalas e inaceptables para un entorno de producción en el modelo `poligpt`. Los tiempos de respuesta escalaron desde 32.82 segundos en la primera consulta hasta un máximo de 136.39 segundos en la cuarta pregunta. Incluso para rechazar la pregunta 5, requirió 118.21 segundos.
- **Rendimiento óptimo de Gemma3:27b:** A pesar de ejecutarse también en el servidor remoto `poligpt`, el modelo `gemma3:27b` mantuvo una estabilidad excelente, con latencias comprendidas entre 2.08 y 4.42 segundos.
- **Modelos locales:** `llama3.2:3b` registró el tiempo de respuesta más rápido de toda la prueba con 1.13 segundos en la pregunta 5.

Evaluación mediante RAGAs

Para validar el rendimiento conceptual y la robustez del asistente RAG DNI de manera automatizada, se ha desarrollado el script `evaluar_ragas.py`, el cual implementa un marco de evaluación basado en el estándar industrial RAGAs (*Retrieval-Augmented Generation Assessment*). Esta metodología complementa el benchmark de rendimiento técnico tradicional mediante un enfoque de «LLM-as-a-Judge» que utiliza el modelo `PoliGPT` configurado a través de `ChatOpenAI(model="poligpt")` como juez central.

ID	Pregunta	Faithfulness	Answer Relevancy	Context Precision	Context Recall
1	¿Qué es DNI?	0.8889	0.6859	0.7500	1.0000
2	¿Cómo me apunto a los desayunos solidarios?	0.7500	0.6879	1.0000	1.0000
3	¿Quién paga la gasolina del refuerzo escolar?	0.5000	0.8012	0.7500	1.0000
4	¿En qué se diferencian los proyectos de RESIS y COLES?	1.0000	0.7629	0.3250	1.0000
5	¿Cuánto cuesta el alquiler de un piso en Valencia?	1.0000	0.0000	0.0000	0.0000

El análisis detallado de las métricas obtenidas tras la ejecución del conjunto de validación revela el siguiente comportamiento en el pipeline del Agente:

- **Recuperación Perfecta en Dominio (Context Recall = 1.0):** Para todas las preguntas relacionadas con la asociación DNI (preguntas 1 a 4), el sistema de recuperación (ChromaDB) encontró exitosamente los fragmentos exactos que contenían la respuesta.
- **Alta Fidelidad General (Faithfulness):** El agente extrae la información de sus fuentes sin inventar datos. Destaca la Pregunta 4, donde la fidelidad es absoluta (1.0), indicando que el modelo de generación se ciñó estrictamente a los textos recuperados. El único bajón se produce en la Pregunta 3 (0.50), posiblemente debido a una excesiva síntesis de la respuesta.
- **Margen de mejora en Precisión de Contexto:** Aunque el sistema encuentra la información correcta (Recall), en ocasiones arrastra *chunks* adicionales con información irrelevante para la consulta específica (Context Precision promedio de 0.56). Un algoritmo de *re-ranking* podría mejorar esta métrica en futuras versiones.
- **Comportamiento Anti-Alucinación (Pregunta 5):** La pregunta trampa sobre el alquiler en Valencia obtiene un 0.0 en Relevancia, Precisión y Recall. Esto representa un éxito total de la política del sistema. Al rechazar responder algo fuera de sus fuentes documentales, RAGAs castiga las métricas de relevancia, pero el sistema obtiene un 1.0 en *Faithfulness* por admitir correctamente que no tiene la información, garantizando cero alucinaciones.

5. Análisis crítico y limitaciones

Un análisis objetivo del comportamiento permite identificar los puntos críticos que condicionan el rendimiento del sistema implementado.

La primera limitación está ligada estrictamente a la recuperación semántica. Al usar una búsqueda top-k fija de $k=5$ en `retriever.py`, el pipeline depende en gran medida de la calidad de los fragmentos recuperados. Al carecer de un mecanismo de *re-ranking* o de un filtrado más fino sobre los *chunks*, si el contexto devuelto contiene ruido, redundancia o información tangencial, el modelo de lenguaje puede perder precisión y activar la lógica de rechazo de `prompts.py`, reduciendo la utilidad de la respuesta final.

En segundo lugar, la capa de gestión de los modelos muestra una fuerte dependencia de la infraestructura utilizada. Los modelos locales ejecutados vía **Ollama** pueden degradar severamente su latencia en función del hardware disponible en la máquina anfitriona. Por el contrario, la ruta remota a través de **PoliGPT** solventa el impacto computacional pero introduce una dependencia adicional de la red y de la disponibilidad del servidor externo; un comportamiento perfectamente visible en la telemetría capturada por `generator.py`.

Por último, existe una desconexión entre la interfaz del contrato y la lógica operativa del agente. Aunque el punto de entrada en `consultar.py` y el orquestador en `pipeline.py` aceptan el parámetro `conversation_id`, el sistema no implementa una memoria conversacional real que conecte los estados de la sesión. Asimismo, el mecanismo de diagnóstico es limitado; de hecho, el campo `trazas` se devuelve por defecto como `None`, lo que restringe la capacidad de auditoría interna ante fallos imprevistos.

En conjunto, estas limitaciones no invalidan el diseño actual del repositorio, pero sí marcan claramente dónde se sitúa el margen de mejora prioritario para consolidar una versión más robusta y preparada para producción.

6. Propuestas de mejora

A partir del comportamiento observado durante el desarrollo y los resultados obtenidos en las fases de análisis, se han identificado diversas líneas de evolución técnica. Estas propuestas están orientadas a optimizar la precisión del agente, sofisticar la arquitectura del repositorio y explotar nuevas capacidades del pipeline RAG en futuras versiones.

Desde la perspectiva del diseño de software, el acoplamiento actual entre los clientes HTTP y la lógica del pipeline se puede sofisticar mediante la migración hacia una arquitectura hexagonal (puertos y adaptadores). El propósito de este cambio estructural en el repositorio es aislar completamente el núcleo de negocio del agente RAG de las tecnologías de infraestructura. Al definir interfaces abstractas (puertos) para el *vector store*, el generador y el generador de *embeddings*, el reemplazo de herramientas como ChromaDB por FAISS, o de Ollama por cualquier otra API externa, se resolvería mediante la simple sustitución de un adaptador, elevando la mantenibilidad del código a estándares profesionales de producción.

Por otra parte, se plantea la necesidad de robustecer el flujo de diagnóstico mediante la automatización de evaluación continua con Ragas, de modo que cualquier cambio en las plantillas de los prompts o en el tamaño de los fragmentos ejecute automáticamente un análisis de fidelidad y tasa de alucinación. Ligado a esto, se propone refinar el campo de trazas del diccionario devuelto en [consultar.py](#) para registrar no solo los tiempos de ejecución, sino el árbol de decisiones interno del pipeline, proporcionando una transparencia absoluta sobre el origen y el procesamiento de cada respuesta generada.

7. Dificultades encontradas y soluciones aplicadas

El desarrollo de este repositorio planteó un conjunto de desafíos técnicos complejos, motivados principalmente por la necesidad de cumplir de forma estricta con el contrato de evaluación automática y de garantizar una estabilidad operativa absoluta bajo infraestructuras heterogéneas. A lo largo del diseño, cada obstáculo se abordó desde una perspectiva de ingeniería, implementando soluciones específicas en el código para asegurar la fiabilidad del agente:

Uno de los principales retos iniciales fue garantizar que el agente pudiera alternar entre la inferencia local con Ollama y el acceso remoto a través de la API de PoliGPT, sin modificar el punto de entrada ni alterar la estructura de salida. Para solucionarlo, se implementó un enrutamiento dual dentro de `generator.py` junto con un mapeo dinámico de alias en `consultar.py`. Esta abstracción permite detectar automáticamente el modelo seleccionado, traducir la consulta al formato específico de cada servidor (Ollama o PoliGPT) y gestionar la conexión de forma transparente para el usuario.